Single Responsibility Principle (SRP)

The **Single Responsibility Principle (SRP)** is the first principle of the **SOLID** design principles.

At a high level:

> **A software component should have one clear responsibility and one primary reason to change.**

SRP is often misunderstood as:

> "A class should have only one method."

That is **not** what SRP means.

A class can have many methods and still follow SRP as long as those methods belong to the same responsibility.

The real question is:

> **Does this component have one cohesive responsibility, or does it contain multiple unrelated reasons to change?**

This README explains:

- Why SRP was needed
- What problem it solves
- What happens when SRP is violated
- How to identify responsibilities
- How to refactor code using SRP
- TypeScript examples
- Go examples
- Real-world backend use cases
- Common mistakes and over-engineering
- How SRP connects with the other SOLID principles

---

1. Introduction

What is SRP?

The Single Responsibility Principle states:

> **A class, module, function, or component should have one responsibility and therefore one primary reason to change.**

The important part is:

```
One primary reason to change
```

Consider this:

```
UserService
    |
    +-- Validate user
    +-- Hash password
    +-- Save user to database
    +-- Send welcome email
    +-- Generate PDF
```

At first, putting everything inside `UserService` might seem convenient.

But now imagine the following changes:

```
Database technology changes
        ↓
UserService changes

Email provider changes
        ↓
UserService changes

PDF format changes
        ↓
UserService changes

Password hashing requirements change
        ↓
UserService changes
```

The same class has multiple independent reasons to change.

That is the problem SRP tries to solve.

A better design could be:

```
UserService
    ↓
Coordinates user registration

UserValidator
    ↓
Validates user data
```

```
PasswordHasher
    ↓
Hashes passwords

UserRepository
    ↓
Persists users

EmailService
    ↓
Sends emails

PdfService
    ↓
Generates PDFs
```

Now each component has a clearer responsibility.

---

2. Why Was SRP Needed?

When applications are small, developers often put several related operations inside one class.

For example:

```
class UserService {
    createUser() {}
}
```

Then requirements are added:

```
class UserService {
    createUser() {}
    validateUser() {}
    hashPassword() {}
    saveUser() {}
    sendEmail() {}
    generateReport() {}
}
```

Eventually:

```
class UserService {
    createUser() {}
    updateUser() {}
    deleteUser() {}
```

```
    validateUser() {}
    hashPassword() {}
    saveUser() {}
    sendEmail() {}
    resetPassword() {}
    generateInvoice() {}
    uploadProfilePicture() {}
    logActivity() {}
}
```

The class slowly becomes responsible for everything.

This usually happens because developers think:

> "All of these things are related to users, so they should be inside `UserService`."

But being related to the same entity does **not** necessarily mean they belong to the same responsibility.

A user can be:

```
User data
Authentication
Persistence
Notifications
Payments
Reports
Files
Analytics
```

These are different concerns even though they all involve a user.

---

3. The Problem

When a component accumulates too many responsibilities, several problems appear.

3.1 God Objects

A **God Object** is a class that knows too much and does too much.

For example:

```
                  UserService
                       |
        +--------------+---------------+
        |              |               |
    Database         Email          Payment
```

```
        |               |               |
     Prisma           SES            Stripe
        |
      Files
        |
     Logging
        |
 Authentication
```

The class becomes the center of many unrelated parts of the system.

The bigger it becomes, the more dangerous changes become.

---

4. Tight Coupling

A class with many responsibilities usually has many dependencies.

For example:

```
import bcrypt from "bcrypt";
import Stripe from "stripe";
import nodemailer from "nodemailer";
import PDFDocument from "pdfkit";
import { PrismaClient } from "@prisma/client";
```

Now `UserService` knows about:

- Password hashing
- Payments
- Email
- PDF generation
- Database persistence

Suppose you replace:

```
SendGrid
```

with:

```
AWS SES
```

The user service must change.

Suppose you replace:

```
Prisma
```

with:

```
MongoDB
```

The user service must change again.

The class is now tightly coupled to infrastructure.

---

5. Testing Nightmares

Imagine:

```
class UserService {
    async registerUser() {
        // validate
        // hash password
        // database
        // email
        // Kafka
        // PDF
    }
}
```

To test the registration flow, you might need:

```
Database
Email provider
Kafka
PDF generator
Password hashing
Configuration
Environment variables
```

A simple unit test becomes difficult because one method is responsible for many external systems.

With SRP:

```
UserValidator
        ↓
Test validation independently
```

```
PasswordHasher
        ↓
Test password hashing independently

UserRepository
        ↓
Test persistence independently

EmailService
        ↓
Test email independently
```

The tests become smaller and more focused.

---

6. Difficult Changes

Imagine the email provider changes.

Without SRP:

```
Email provider changes
        ↓
Large UserService changes
        ↓
Many unrelated dependencies
        ↓
Large regression-testing surface
```

With SRP:

```
Email provider changes
        ↓
EmailService changes
```

The change is isolated.

This is sometimes called reducing the **blast radius** of a change.

---

7. Difficult to Understand

Imagine opening a 2,000-line service.

You want to understand how user registration works, but the same class contains:

```
Authentication
Payments
Email
Database
File uploads
Analytics
PDF generation
Logging
```

You now have to understand unrelated code before changing a small feature.

SRP encourages components with clear boundaries:

```
UserService
UserRepository
EmailService
PasswordHasher
UserValidator
```

The code becomes easier to navigate.

---

8. The Solution

SRP encourages us to separate responsibilities according to their **reason for change**.

Instead of:

```
UserService
    |
    +-- Validation
    +-- Hashing
    +-- Database
    +-- Email
    +-- PDF generation
```

we create:

```
UserService
    |
    +-- UserValidator
    +-- PasswordHasher
    +-- UserRepository
```

```
    +-- EmailService
    +-- PdfService
```

The `UserService` can still coordinate the complete workflow.

The difference is that it no longer owns the implementation of every concern.

---

9. What SRP Improves

Maintainability

Changes are more localized.

```
Change database
      ↓
UserRepository

Change email provider
      ↓
EmailService

Change password hashing
      ↓
PasswordHasher
```

Testability

Each component can be tested independently.

For example:

```
PasswordHasher
    ↓
Test password hashing

UserRepository
    ↓
Test persistence

EmailService
    ↓
Test email behavior
```

The tests no longer need every dependency at once.

Readability

Compare:

```
UserService
    1500 lines
```

with:

```
UserService
UserRepository
EmailService
PasswordHasher
UserValidator
```

The second design communicates intent more clearly.

Lower Coupling

Instead of:

```
Large Service
   |
   +-- Database
   +-- Email
   +-- Payment
   +-- File system
   +-- PDF
```

you move toward:

```
UserService
    |
    +-- UserRepository
    +-- EmailService
```

The service has fewer direct responsibilities.

---

10. Important Clarification: One Responsibility ≠ One Method

This is one of the most important points about SRP.

This is perfectly reasonable:

```
class UserRepository {
    create() {}

    findById() {}

    findByEmail() {}

    update() {}

    delete() {}
}
```

There are several methods.

But all methods share one responsibility:

> **Persisting users.**

Therefore, the class can still follow SRP.

Do not think:

```
One method
    ↓
One class
```

Think:

```
One cohesive responsibility
    ↓
One primary reason to change
```

---

11. Implementation Guide

The easiest way to apply SRP is to follow a step-by-step reasoning process.

Step 1: Understand What the Component Does

Take a large class and list everything it currently does.

Example:

```
UserService

- Validate email
- Hash password
- Save user
- Send email
- Generate PDF
- Log activity
```

Don't immediately start moving code.

First understand the existing responsibilities.

Step 2: Group Related Behavior

Group operations that belong to the same concern.

```
Validation
    └── Validate user

Security
    └── Hash password

Persistence
    └── Save user

Notification
    └── Send email

Reporting
    └── Generate PDF

Observability
    └── Log activity
```

This reveals natural responsibility boundaries.

Step 3: Ask "What Would Cause This to Change?"

This is one of the best SRP questions.

For example:

```
Database requirements change
        ↓
Persistence logic changes

Email provider changes
        ↓
Email logic changes

Security requirements change
        ↓
Password hashing changes
```

If all these changes affect one class, the class probably has multiple responsibilities.

Step 4: Extract Independent Responsibilities

Create components around meaningful boundaries.

For example:

```
UserService
    |
    +-- UserValidator
    +-- PasswordHasher
    +-- UserRepository
    +-- EmailService
```

Step 5: Keep the Main Workflow Simple

After extraction, the main service should coordinate the use case.

For example:

```
class UserService {
    async registerUser() {
        validate();
        hashPassword();
        saveUser();
```

```
        sendEmail();
    }
}
```

This is not automatically a violation.

If the responsibility of the service is:

> **Coordinate the user registration use case**

then calling multiple focused components is completely reasonable.

Step 6: Avoid Over-Engineering

Do not create a separate class for every small line of code.

Bad:

```
EmailValidationService
EmailValidationFactory
EmailValidationManager
EmailValidationProvider
EmailValidationStrategy
```

for:

```
email.includes("@")
```

SRP is about clear boundaries, not maximum fragmentation.

---

12. TypeScript Example

Before — Violating SRP

Consider a user registration service:

```
import bcrypt from "bcrypt";

class UserService {
    async registerUser(
        name: string,
        email: string,
        password: string
```

```
    ) {
        // Validation
        if (!email.includes("@")) {
            throw new Error("Invalid email");
        }

        // Password hashing
        const hashedPassword =
            await bcrypt.hash(password, 10);

        // Database logic
        const user = await db.user.create({
            data: {
                name,
                email,
                password: hashedPassword
            }
        });

        // Email notification
        await sendEmail({
            to: email,
            subject: "Welcome",
            body: "Welcome to our application"
        });

        return user;
    }
}
```

The class performs:

```
UserService
    |
    +-- Validation
    +-- Password hashing
    +-- Database persistence
    +-- Email notification
```

There are multiple reasons for this class to change.

---

13. TypeScript — After SRP

UserValidator

```
class UserValidator {
    validateEmail(email: string): void {
        if (!email.includes("@")) {
            throw new Error("Invalid email");
        }
    }
}
```

Its responsibility is:

```
Validate user input
```

PasswordHasher

```
import bcrypt from "bcrypt";

class PasswordHasher {
    async hash(password: string): Promise<string> {
        return bcrypt.hash(password, 10);
    }
}
```

Its responsibility is:

```
Password hashing
```

UserRepository

```
interface User {
    id: string;
    name: string;
    email: string;
    password: string;
}

interface CreateUserInput {
    name: string;
```

```
    email: string;
    password: string;
}

class UserRepository {
    async create(
        data: CreateUserInput
    ): Promise<User> {
        return db.user.create({
            data
        });
    }
}
```

Its responsibility is:

```
User persistence
```

EmailService

```
class EmailService {
    async sendWelcomeEmail(
        email: string
    ): Promise<void> {
        await sendEmail({
            to: email,
            subject: "Welcome",
            body: "Welcome to our application"
        });
    }
}
```

Its responsibility is:

```
Sending email
```

UserService

Now the main service coordinates the workflow:

```
class UserService {
    constructor(
        private readonly validator: UserValidator,
```

```
        private readonly passwordHasher: PasswordHasher,
        private readonly userRepository: UserRepository,
        private readonly emailService: EmailService
    ) {}

    async registerUser(
        name: string,
        email: string,
        password: string
    ) {
        this.validator.validateEmail(email);

        const hashedPassword =
            await this.passwordHasher.hash(password);

        const user =
            await this.userRepository.create({
                name,
                email,
                password: hashedPassword
            });

        await this.emailService.sendWelcomeEmail(
            email
        );

        return user;
    }
}
```

The responsibility of `UserService` is now:

```
Coordinate user registration
```

It does not implement:

```
How validation works
How password hashing works
How persistence works
How email delivery works
```

---

14. TypeScript — Architecture

The final structure looks like:

```
                  UserService
                       |
        +--------------+--------------+
        |              |              |
        ↓              ↓              ↓
 UserValidator   PasswordHasher   UserRepository

                       +
                       |
                       ↓
                  EmailService
```

Or conceptually:

```
UserService
    ↓
Application workflow

UserValidator
    ↓
Validation

PasswordHasher
    ↓
Security

UserRepository
    ↓
Persistence

EmailService
    ↓
Notification
```

---

15. TypeScript — Benefits of the Refactoring

Suppose the application moves from:

```
bcrypt
```

to:

```
argon2
```

Only:

```
PasswordHasher
```

needs to change.

Suppose:

```
PostgreSQL + Prisma
```

is replaced by:

```
MongoDB
```

The persistence implementation changes.

Suppose:

```
SendGrid
```

is replaced by:

```
AWS SES
```

The email implementation changes.

The registration workflow does not need to be rewritten.

This is the practical value of SRP.

---

16. Go Example

We can apply the same principle in Go.

Suppose we have an order-processing system.

The processor performs:

- price calculation,
- persistence,
- logging.

---

17. Go — Before SRP

```go
package order

import (
    "database/sql"
    "log"
)

type Order struct {
    ID       string
    Quantity int
    Price    float64
}

type OrderProcessor struct {
    DB *sql.DB
}

func (p *OrderProcessor) ProcessOrder(
    order Order,
) error {

    // Calculate total
    total := float64(order.Quantity) * order.Price

    // Save order
    _, err := p.DB.Exec(
        "INSERT INTO orders (id, total) VALUES ($1, $2)",
        order.ID,
        total,
    )

    if err != nil {
        return err
    }

    // Logging
    log.Printf(
        "Order %s processed with total %.2f",
        order.ID,
        total,
    )

    return nil
}
```

The processor owns three concerns:

```
OrderProcessor
    |
    +-- Calculation
    +-- Database persistence
    +-- Logging
```

---

18. Go — After SRP

Calculator

```go
package order

type Calculator struct{}

func (Calculator) CalculateTotal(
    quantity int,
    price float64,
) float64 {
    return float64(quantity) * price
}
```

Responsibility:

```
Calculate order totals
```

Repository

Define an interface:

```go
package order

type Repository interface {
    Save(order Order) error
}
```

Its responsibility is:

```
Persist orders
```

A PostgreSQL implementation:

```go
package order

import "database/sql"

type PostgresRepository struct {
    DB *sql.DB
}

func (r *PostgresRepository) Save(
    order Order,
) error {

    _, err := r.DB.Exec(
        "INSERT INTO orders (id, total) VALUES ($1, $2)",
        order.ID,
        order.Total,
    )

    return err
}
```

Logger

Define a focused interface:

```go
package order

type Logger interface {
    Info(message string)
}
```

Implementation:

```go
package order

import "log"

type StandardLogger struct{}

func (StandardLogger) Info(
    message string,
) {
    log.Println(message)
}
```

---

19. Go — OrderProcessor After SRP

```go
package order

import "fmt"

type OrderProcessor struct {
    calculator Calculator
    repository Repository
    logger     Logger
}

func NewOrderProcessor(
    calculator Calculator,
    repository Repository,
    logger Logger,
) *OrderProcessor {

    return &OrderProcessor{
        calculator: calculator,
        repository: repository,
        logger:     logger,
    }
}

func (p *OrderProcessor) ProcessOrder(
    order Order,
) error {

    order.Total =
        p.calculator.CalculateTotal(
            order.Quantity,
            order.Price,
        )

    if err := p.repository.Save(order); err != nil {
        return err
    }

    p.logger.Info(
        fmt.Sprintf(
            "Order %s processed with total %.2f",
            order.ID,
            order.Total,
        ),
    )

    return nil
}
```

Now:

```
OrderProcessor
      |
      +-- Calculator
      |
      +-- Repository
      |
      +-- Logger
```

The processor is responsible for:

> **Coordinating the order-processing workflow.**

---

20. Go — Why Interfaces Are Useful

Because the processor depends on interfaces:

```go
type Repository interface {
    Save(order Order) error
}
```

we can have multiple implementations:

```
Repository
    |
    +-- PostgresRepository
    +-- MongoRepository
    +-- InMemoryRepository
    +-- MockRepository
```

The `OrderProcessor` does not care which persistence technology is used.

This is especially useful for testing.

---

21. Go — Testing with a Mock

We can create a simple mock:

```go
type MockRepository struct {
    SavedOrder Order
}
```

```
func (m *MockRepository) Save(
    order Order,
) error {

    m.SavedOrder = order

    return nil
}
```

Now the processor can be tested without a real PostgreSQL database.

The test can focus on:

```
Did OrderProcessor calculate the correct total?

Did OrderProcessor call the repository?

Did OrderProcessor log the result?
```

The database implementation does not need to exist during the unit test.

---

22. Before vs After

TypeScript

Before

```
UserService
    |
    +-- Validation
    +-- Password hashing
    +-- Database
    +-- Email
```

After

```
UserService
    |
    +-- UserValidator
    +-- PasswordHasher
    +-- UserRepository
    +-- EmailService
```

Go

Before

```
OrderProcessor
    |
    +-- Calculation
    +-- Database
    +-- Logging
```

After

```
OrderProcessor
    |
    +-- Calculator
    +-- Repository
    +-- Logger
```

The important improvement is not the number of classes or files.

The important improvement is:

```
Different responsibilities
        ↓
Clear ownership
        ↓
Independent changes
        ↓
Lower coupling
```

---

23. Real-World Use Cases

SRP becomes particularly useful in backend systems where multiple concerns exist.

Common examples include:

```
HTTP
Business logic
Database
Caching
Authentication
Authorization
Payments
Email
```

```
File storage
Messaging
Logging
External APIs
```

23.1 E-Commerce Checkout

Checkout may involve:

```
Cart validation
Pricing
Coupons
Inventory
Order creation
Payment
Email
Events
Invoices
```

Instead of one giant service:

```
CheckoutService
```

you may have:

```
CheckoutService
    |
    +-- CartValidator
    +-- PricingService
    +-- InventoryService
    +-- PaymentService
    +-- OrderRepository
    +-- NotificationService
```

`CheckoutService` coordinates the checkout use case.

The individual components own their specific concerns.

---

24. Hotel Booking System

A booking system may involve:

```
Availability checking
Room locking
Price calculation
Booking creation
Payment
Notifications
Events
```

A reasonable design could be:

```
BookingService
    |
    +-- AvailabilityService
    +-- RoomLockService
    +-- PricingService
    +-- BookingRepository
    +-- PaymentService
    +-- NotificationService
```

This becomes especially useful when dealing with:

- database transactions,
- concurrent bookings,
- row locking,
- distributed systems,
- payment providers,
- asynchronous events.

---

25. Payment Systems

A payment system could involve multiple providers:

```
PaymentService
        |
        ↓
PaymentProvider
      /
     /
 Stripe   Razorpay
```

Each provider handles its own integration details.

The application service does not need to know every API detail.

---

26. File Storage Systems

A file-storage application may include:

```
File validation
S3 upload
Metadata persistence
Virus scanning
Quota calculation
Notifications
```

These concerns can be separated:

```
StorageService
    |
    +-- FileValidator
    +-- ObjectStorage
    +-- FileRepository
    +-- QuotaService
    +-- NotificationService
```

---

27. When Should You Use SRP?

SRP is particularly valuable when:

- A class is becoming very large.
- Unrelated requirements frequently modify the same class.
- Unit tests are difficult to write.
- The class depends on many unrelated technologies.
- Business logic is mixed with infrastructure logic.
- A change has a large blast radius.
- Different teams frequently modify the same component.
- Several different reasons can cause the component to change.

One strong signal is:

> **"Every new feature keeps adding another unrelated method to the same class."**

That is usually worth investigating.

---

28. Common Pitfalls

SRP is useful, but it can also be overused.

28.1 One Method = One Class

This is a common mistake:

```
CreateUserService
GetUserService
UpdateUserService
DeleteUserService
```

For a small application, this can create unnecessary complexity.

A simpler design might be:

```
UserService
    |
    +-- create()
    +-- get()
    +-- update()
    +-- delete()
```

These operations can reasonably belong to the same user-management responsibility.

28.2 Too Many Interfaces

Do not create interfaces simply because you want to "follow SOLID."

Interfaces are especially useful when you have:

```
Multiple implementations
Replaceable infrastructure
External providers
Testing requirements
```

An abstraction should solve a real design problem.

28.3 Splitting Code Without Understanding It

Do not take:

```
UserService
    1000 lines
```

and mechanically turn it into:

```
UserService1
UserService2
UserService3
UserService4
```

The number of files decreased, but the architecture may not have improved.

The responsibilities need to be understood first.

28.4 Creating Abstractions Too Early

Avoid designing:

```
EmailValidationFactory
EmailValidationProvider
EmailValidationManager
EmailValidationStrategy
```

for simple validation logic.

For example:

```
function isValidEmail(email: string): boolean {
    return email.includes("@");
}
```

may be all you need.

Start simple.

Introduce abstractions when the system actually needs them.

---

29. SRP and Cohesion

A useful concept for understanding SRP is **cohesion**.

Cohesion asks:

> **How closely related are the responsibilities inside a component?**

High cohesion:

```
class UserRepository {
    create() {}

    findById() {}

    findByEmail() {}

    update() {}

    delete() {}
}
```

All methods relate to:

```
User persistence
```

Low cohesion:

```
class UserService {
    createUser() {}

    sendEmail() {}

    generatePDF() {}

    processPayment() {}

    uploadFile() {}
}
```

These operations belong to different concerns.

A healthy design generally aims for:

```
High Cohesion
     +
Low Coupling
```

SRP helps achieve that.

---

30. SRP and the Other SOLID Principles

SRP is the first SOLID principle:

```
S → Single Responsibility Principle
O → Open/Closed Principle
L → Liskov Substitution Principle
I → Interface Segregation Principle
D → Dependency Inversion Principle
```

These principles are related.

For example:

SRP

Separates responsibilities:

```
UserService
UserRepository
EmailService
```

Dependency Inversion

Allows us to depend on abstractions:

```
interface UserRepository {
    create(): Promise<User>;
}
```

Open/Closed Principle

Makes it easier to add implementations without constantly changing existing code:

```
PaymentProvider
    |
    +-- StripeProvider
    +-- RazorpayProvider
```

Interface Segregation

Encourages focused interfaces:

```
Logger
    |
    +-- info()

Repository
```

```
    |
    +-- save()
```

Good application architecture often emerges when these principles work together.

---

31. Practical SRP Decision Process

Whenever you see a large component, ask these questions:

```
1. What does this component do?

        ↓

2. What responsibilities does it contain?

        ↓

3. Which responsibilities are related?

        ↓

4. Which responsibilities can change independently?

        ↓

5. Who would request those changes?

        ↓

6. Can those responsibilities have clearer ownership?

        ↓

7. Would extracting them actually improve the design?

        ↓

8. Am I separating code for a real reason,
   or simply creating more classes?
```

This approach is more useful than blindly following a rule.

---

32. SRP Checklist

Use this during code review:

```
[ ] Can I clearly describe this component's responsibility?

[ ] Does it contain unrelated behavior?

[ ] Could unrelated requirements cause it to change?

[ ] Does it depend on many unrelated technologies?

[ ] Is it difficult to unit test?

[ ] Is business logic mixed with infrastructure logic?

[ ] Can one responsibility change without touching another?

[ ] Would extracting a responsibility make the code clearer?

[ ] Am I avoiding unnecessary fragmentation?

[ ] Is the abstraction solving an actual problem?
```

---

33. Final Mental Model

Do not remember SRP as:

```
One class
    ↓
One method
```

Remember it as:

```
One cohesive responsibility
          ↓
One primary reason to change
          ↓
Clear ownership
          ↓
Lower coupling
          ↓
Easier testing
          ↓
Easier maintenance
```

The best question to ask is:

> **"What different kinds of changes could force this component to change?"**

For example:

```
Database changes
Email changes
Payment changes
Security changes
Reporting changes
```

If all of them modify the same class, that class probably has too many responsibilities.

But if the answer is:

```
Changes to the user-registration workflow
```

then the responsibility is much more focused.

---

34. Conclusion

The **Single Responsibility Principle** is fundamentally about creating **clear responsibility boundaries**.

It does not mean:

- every class must be tiny,
- every method needs its own class,
- every class needs an interface,
- every file must contain only a few lines.

Instead, SRP asks us to design components so that:

```
Related behavior
       ↓
Lives together

Unrelated behavior
       ↓
Lives separately
```

A good SRP-based design generally gives us:

```
Clear responsibilities
        ↓
High cohesion
        ↓
Lower coupling
```

```
        ↓
Smaller changes
        ↓
Simpler tests
        ↓
Better maintainability
        ↓
Easier long-term evolution
```

The most important lesson is:

> **SRP is not about making classes small. It is about making their responsibilities clear.**

When a component has one cohesive responsibility and one primary reason to change, the code becomes easier to understand, test, modify, and maintain.

Quick Reference

```
┌──────────────────────────────────────────────┐
│      SINGLE RESPONSIBILITY PRINCIPLE         │
├──────────────────────────────────────────────┤
│                                              │
│  One cohesive responsibility                 │
│              ↓                               │
│  One primary reason to change                │
│              ↓                               │
│  Clear ownership                             │
│              ↓                               │
│  Lower coupling                              │
│              ↓                               │
│  Easier testing                              │
│              ↓                               │
│  Easier maintenance                          │
│                                              │
└──────────────────────────────────────────────┘
```

> **Ask yourself: "Does this component have one cohesive responsibility and one primary reason to change?"**

That question is the heart of SRP.